



Date: 09 / 08 / 2026

Lab Practical #08:

Study Packet capture and header analysis by Wireshark (HTTP, TCP, UDP, IP, etc.)

Practical Assignment #08:

1. Give details about usage of Wireshark tool.

Wireshark is a free, open-source network protocol analyzer used to capture and inspect the traffic passing through a network interface in real time. It's widely used by network administrators, security professionals, and developers for troubleshooting, security analysis, and protocol learning.

Core Uses

- **Network troubleshooting** — diagnosing slow connections, dropped packets, DNS failures, or misconfigured devices
- **Security analysis** — detecting suspicious traffic, unauthorized connections, or malware communication patterns (defensive/monitoring use)
- **Protocol learning** — seeing exactly how protocols like TCP, HTTP, DNS, and TLS behave on the wire
- **Application debugging** — verifying that software is sending/receiving data as expected

Basic Workflow

1. **Select an interface** — On launch, Wireshark lists available network interfaces (Wi-Fi, Ethernet, loopback, etc.). Choose the one carrying the traffic you want to inspect.
2. **Start capturing** — Click the interface or the shark-fin icon to begin. Packets appear in real time in the top pane.
3. **Stop the capture** — Click the red square icon once you've gathered enough data.
4. **Analyze packets** — The interface has three main panes:
 - **Packet list** (top): one row per packet, with time, source, destination, protocol, and info
 - **Packet details** (middle): expandable tree showing each protocol layer (Ethernet → IP → TCP → application layer)
 - **Packet bytes** (bottom): raw hex/ASCII view of the packet

Filtering Traffic

Wireshark's power comes from its filtering syntax, applied in the **display filter bar** at the top:

Filter	Purpose
ip.addr == 192.168.1.10	Show traffic to/from a specific IP
tcp.port == 443	Show HTTPS traffic
http	Show only HTTP packets
dns	Show only DNS queries/responses
tcp.flags.syn == 1	Show TCP connection attempts
http.request.method == "GET"	Show only GET requests

2. Packet capture and header analysis by Wireshark (HTTP, TCP, UDP, IP, etc.)

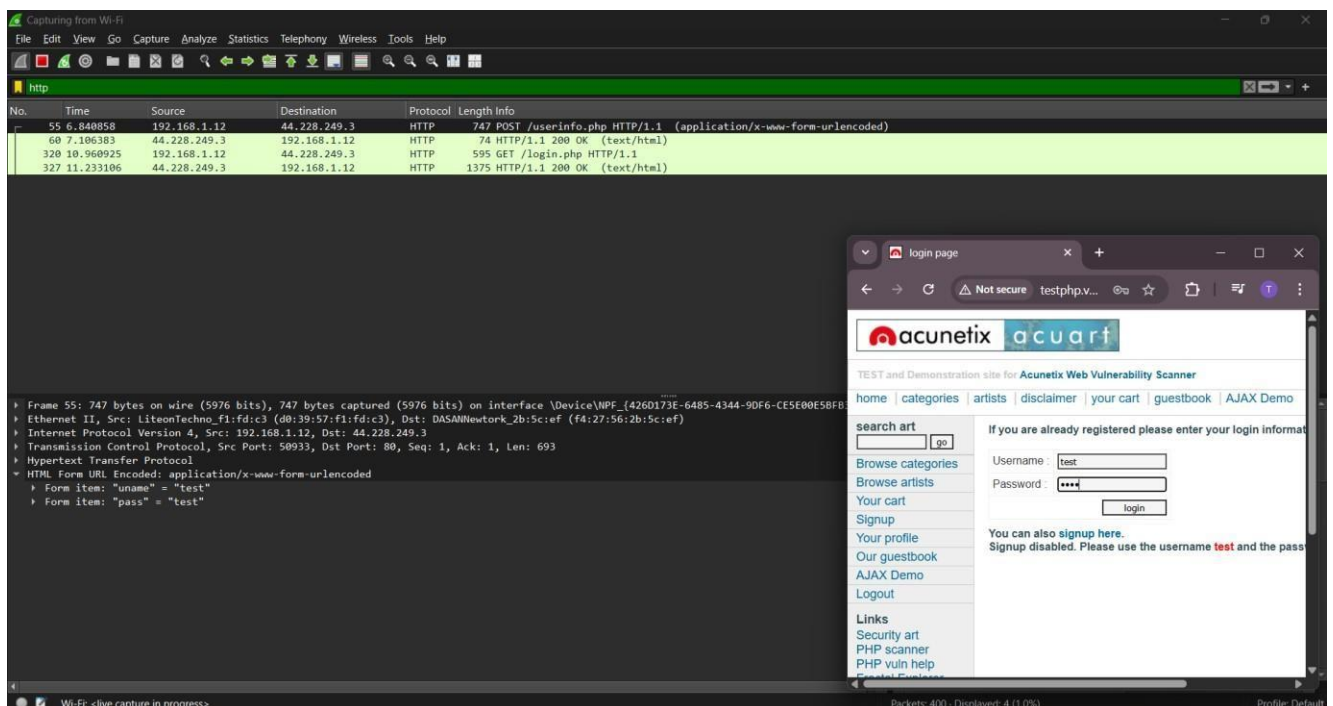
1) Study Packet capture and header analysis by wire-shark.

a) Capturing and observe Live Traffic – Use any website.

2) Applying Capture Filters:

a) HTTP traffic: HTTP

- For HTTP - Inspect request and response headers, URLs, and status codes.
- For HTTPS, observe the handshake process (TLS).

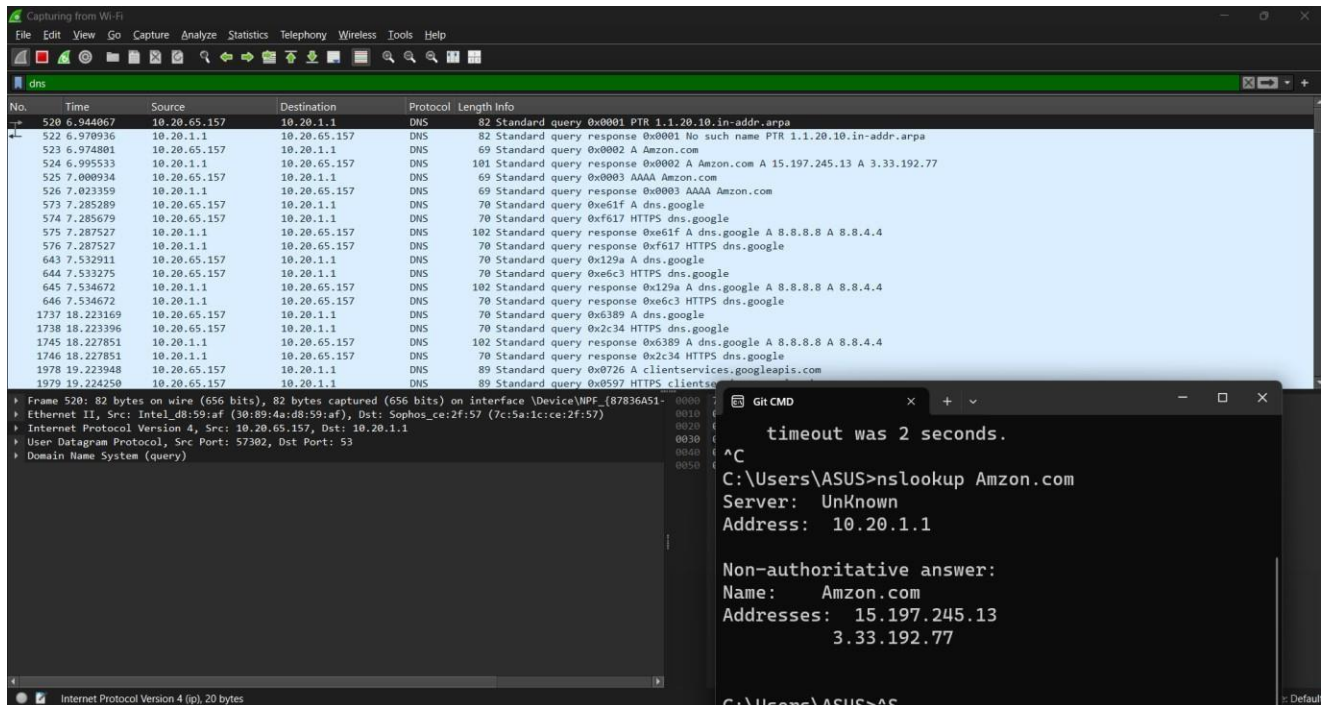


The screenshot shows Wireshark capturing traffic from Wi-Fi. The packet list shows several HTTP packets. The packet details pane shows the structure of an HTTP POST request to /userinfo.php. The packet bytes pane shows the raw data. A browser window in the background shows the Acunetix website login page.

Date: 09 / 08 / 2026

b) DNS requests: DNS

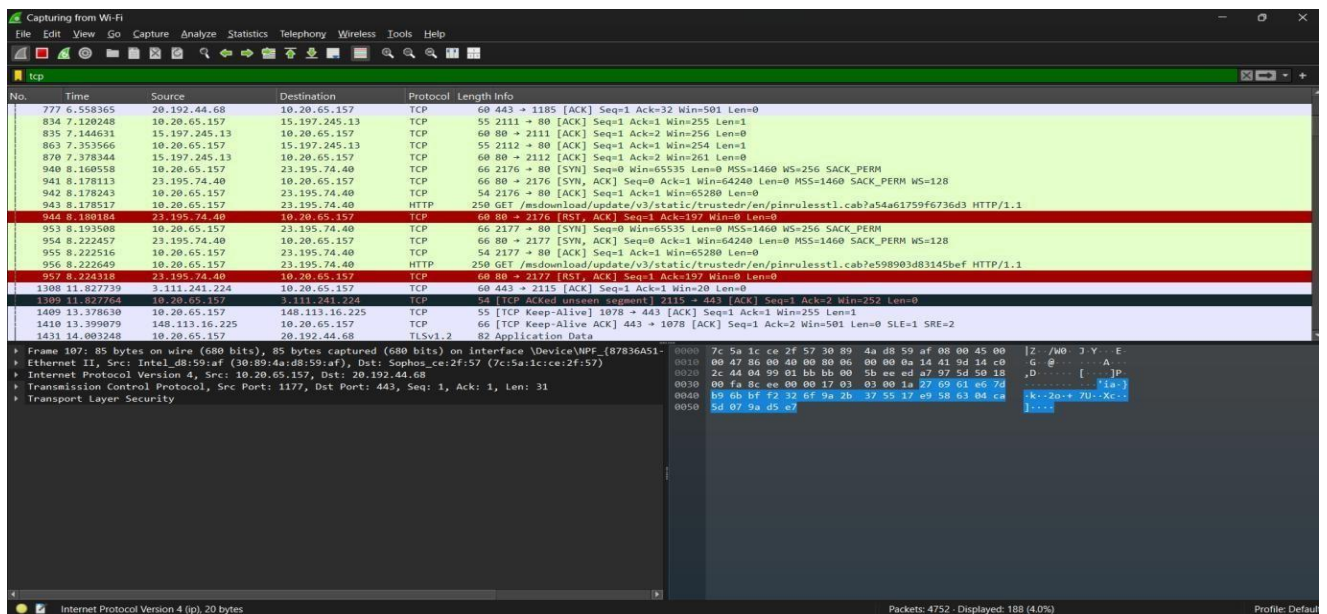
- i) Observe how domain names are translated into IP addresses.
- ii) Identify DNS request and response packets.



The image shows a Wireshark packet capture of DNS traffic. The packet list shows several DNS queries and responses. The packet details pane shows the structure of a DNS query packet. To the right, a terminal window shows the output of the command `nslookup Amazon.com`, which returns the IP address 10.20.1.1 and a non-authoritative answer from the local DNS server.

c) TCP packets: TCP

- i) Observe the 3-way handshake (SYN, SYN-ACK, ACK).
- ii) Identify sequence numbers and acknowledgment numbers.

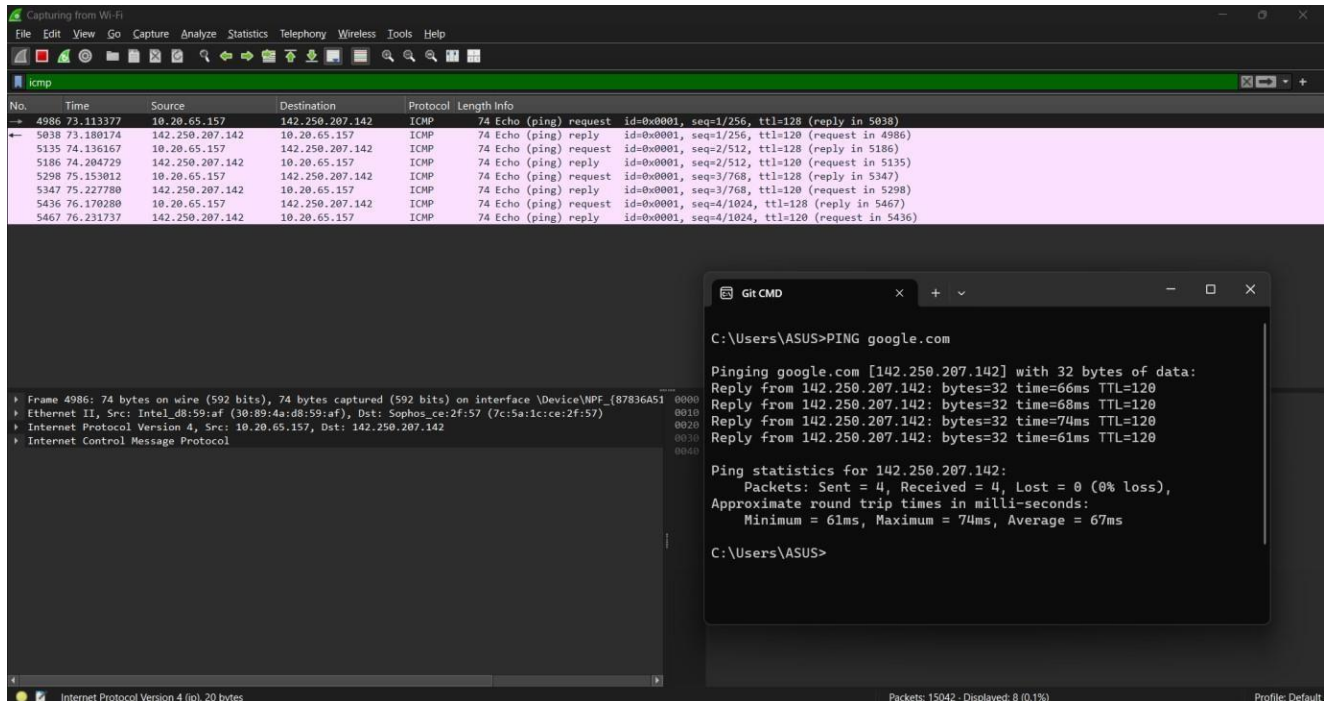


The image shows a Wireshark packet capture of TCP traffic. The packet list shows a 3-way handshake (SYN, SYN-ACK, ACK) and subsequent data transfer. The packet details pane shows the structure of a TCP packet. The packet bytes pane shows the raw data of the packets.

Date: 09 / 08 / 2026

d) ICMP (ping): ICMP

- i) Send ping requests (ping google.com) and observe request/reply packets.



The screenshot displays two windows. The top window is Wireshark, showing a packet capture of ICMP Echo (ping) requests and replies between 10.20.65.157 and 142.250.207.142. The bottom window is a Git CMD terminal showing the output of the command 'PING google.com'.

Wireshark Packet Capture:

No.	Time	Source	Destination	Protocol	Length	Info
4986	73.113377	10.20.65.157	142.250.207.142	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (reply in 5038)
5038	73.180174	142.250.207.142	10.20.65.157	ICMP	74	Echo (ping) reply id=0x0001, seq=1/256, ttl=128 (request in 4986)
5135	74.136167	10.20.65.157	142.250.207.142	ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (reply in 5186)
5186	74.204729	142.250.207.142	10.20.65.157	ICMP	74	Echo (ping) reply id=0x0001, seq=2/512, ttl=128 (request in 5135)
5298	75.153012	10.20.65.157	142.250.207.142	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 5347)
5347	75.227780	142.250.207.142	10.20.65.157	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=128 (request in 5298)
5436	76.170280	10.20.65.157	142.250.207.142	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 5467)
5467	76.231737	142.250.207.142	10.20.65.157	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=128 (request in 5436)

Git CMD Output:

```
C:\Users\ASUS>PING google.com

Pinging google.com [142.250.207.142] with 32 bytes of data:
Reply from 142.250.207.142: bytes=32 time=66ms TTL=128
Reply from 142.250.207.142: bytes=32 time=68ms TTL=128
Reply from 142.250.207.142: bytes=32 time=74ms TTL=128
Reply from 142.250.207.142: bytes=32 time=61ms TTL=128

Ping statistics for 142.250.207.142:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 61ms, Maximum = 74ms, Average = 67ms

C:\Users\ASUS>
```